Assignment 1: Polynomial Calculator

Data Structures (CS-UH 1050)

1. Code of Conduct

All assignments are graded, meaning we expect you to adhere to the academic integrity standards of NYU Abu Dhabi. To avoid any confusion regarding this, we will briefly state what is and isn't allowed when working on an assignment.
Any documents and program code that you submit must be fully written by you. You can discuss your work with fellow students, as long as these discussions are restricted to general solution techniques. Put differently, these discussions should not be about the concrete code you are writing, nor about specific results you wish to submit. When discussing an assignment with others, this should never lead to you possessing the complete or partial solution of others, regardless of whether the solution is in paper or digital form, and independent of who made the solution, this includes solutions from previous years, other courses, other universities, or any code obtained from the Internet. This also implies that there is never a valid reason to share your code with fellow students, and that there is no valid reason to publish your code online in any form. Every student is responsible for the work they submit. If there is any doubt during the grading about whether a student created the assignment themselves (e.g., if the solution matches that of others), we reserve the option to question the student about their submitted work and/or ask for a demonstration. Potential academic integrity violation cases are handled by the Office of Undergraduate Academic Policies according to the Academic Integrity Policy and Procedure at New York University Abu Dhabi (see https://students.nyuad.nyu.edu/campus-life/community-standards/policies/academic-integrity/).

Course policy on ChatGPT and AI tools: It is important that the written work required by the course is yours. You may not use ChatGPT or any other AI tools to generate program code for this assignment.

2. Introduction

In this assignment, you will develop a Polynomial Calculator using C++ that will be used to perform addition, multiplication, evaluation, equality checking, and determination of the degree of two single-variable polynomials.

You are supposed to create a program using the following concepts covered in class:

- File input/output in C++
- Object Oriented Programming
- Linked Lists

Note: You are not allowed to use any built-in STL data structures such as list, vector, etc.

3. Polynomial Representation Rules

A polynomial is represented as a sequence of terms of the form

<**coef**>x^<**exp**>

- ***coef*** is a signed integer (e.g., -5, +3, 7). The positive sign for the first term of a polynomial expression is optional.
- ***exp*** is a non-negative integer without a sign (0, 1, 2, ...).
- The variable is always the lowercase letter **x**.

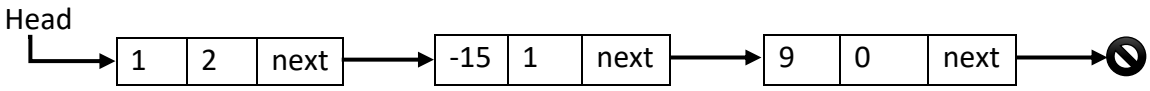
Following are some examples of valid polynomial expressions:

- 12x^1 +3x^0
- +2x^1 -1x^2 +5x^0
- -2x^2 +2x^2 +3x^1 -4x^0
- 2x^1

Following are some examples of invalid polynomial expressions:

- 2x^12 x^3 (No operator between two terms)
- 2x^ (Exponent is missing)
- 2x (Exponent is missing)
- x^2 (Coefficient is missing)
- x (Coefficient and exponent are missing)
- 2x^1 +4^x0+ (The expression is not properly terminated)
- 3x^1 -- 3x^0 (Multiple operators next to each other)

A polynomial expression can be stored in a linked list by representing each term as a node of the linked list. Each node stores the coefficient and exponent of one term of the polynomial. For example, the expression $x^2 - 15x + 9$ can be stored in a linked list as follows:



The linked list storing a polynomial **must always satisfy**:

- Terms are stored in **strictly decreasing order of exponent**
- No two nodes may have the same exponent

4. Implementation

You will be provided with a starter code containing class declarations and the main function. Your task is to complete the missing methods of the given classes. You may add new helper methods in the classes, but you are not allowed to change/remove the headers of existing methods. The input/output format of the program should be the same as illustrated in the screenshots.

Method Description of <u>LinkedList class</u>:

1. LinkedList::LinkedList()

The constructor for the LinkedList class should create an empty linked list.

2. void LinkedList::insert(int coef, int expo)

This method should be used to insert a new term of the polynomial expression into the linked list while also simplifying the polynomial. Terms must always be stored in strictly decreasing (descending) order of exponent (highest exponent first). For example, if the polynomial expression +3x^2 +4x^3 +2x^2 -6x^1 +4x^0 is being inserted into the LinkedList (one term at a time), the *insert* method should populate a linked list containing the simplified and sorted form of the same expression, i.e., +4x^3 +5x^2 -6x^1 +4x^0.

3. void LinkedList::print(ostream &os) const

This method will print a linked list in the following format on a specific output stream (os) e.g., on the standard output stream (`cout`) or to a given file object:

```
+4x^3 +2x^2 +3x^1 +8x^0
```

The print() method of the LinkedList class can be reused/called by other methods e.g. from the display() methods of PolyCalculator class.

4. bool LinkedList::operator==(const LinkedList &other) const

This method should check if the current linked list is identical to another linked list that has been passed as an argument to the method.

5. void LinkedList::removeAll()

This method should remove all nodes of the linked list object for which it is called. It can be used to clear the contents of a linked list so that it can be reused in the same program session.

6. LinkedList::~LinkedList()

This destructor of the LinkedList class should remove all nodes of the linked list object. Make sure to delete/clear all dynamically allocated memory.

Method Description of <u>PolyCalculator class</u>:

1. void PolyCalculator::input()

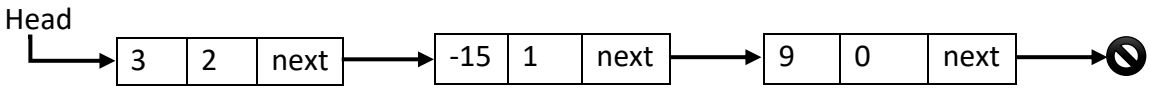
This method should prompt the user to enter two polynomial expressions, as illustrated in the screenshot of the terminal that invokes this method. The existing linked lists/expressions must be cleared before storing the new data in them. The terms in the user input may appear in any order. Duplicate exponents may appear in the input expression. However, the linked list must not contain multiple nodes with the same exponent. When a term with an existing exponent is inserted, the coefficients must be combined into a single node by the ***insert()*** method.

```
>input
Enter first Polynomial expression: +4x^3 +2x^2 +3x^1 +8x^0
Enter second Polynomial expression: +5x^3 +1x^2 +6x^0
```

2. bool PolyCalculator::parse(string str, LinkedList& list)

This method takes a polynomial expression stored in a string and a linked list object as arguments. It should parse the expression and store the resulting terms in the given linked list. The method must verify whether the provided string represents a valid polynomial expression. If the expression is invalid, the linked list must be cleared and the method must return `false`.

The method should identify, extract, and process each term of the polynomial expression in the given string and insert it into the linked list by invoking the insert() method of the LinkedList class. The parse() function must only extract terms and insert them into the linked list. It must NOT perform sorting or simplification. Sorting and combining like terms must be handled exclusively by the insert() method. For example, if the expression 1x^2 -15x^1 +9x^0 +2x^2 is passed to the ***parse()*** function along with a linked list object, the content of the linked list should be as follows:



The ***parse()*** method should return `true` if the expression is valid and `false` otherwise.

3. void PolyCalculator::display()

This method should display the two currently stored polynomial expressions as illustrated below.

```
>display
Exp1: +4x^3 +2x^2 +3x^1 +8x^0
Exp2: +5x^3 +1x^2 +6x^0
```

4. void PolyCalculator::add()

This method should add the two polynomial expressions previously loaded to the calculator (i.e., the values of list1 and list2) and print out the result as illustrated below. The result should be stored in the list3.

```
>add
Exp1 + Exp2 = +9x^3 +3x^2 +3x^1 +14x^0
```

5. void PolyCalculator::sub()

This method should subtract the second polynomial from the first polynomial (previously loaded to the calculator) and print out the result as illustrated below. The result should be stored in the linked list list3.

```
>sub
Exp1 - Exp2 = -1x^3 +1x^2 +3x^1 +2x^0
```

6. void PolyCalculator::mul()

This method should multiply the two polynomial expressions previously loaded to the calculator (i.e., the values of attributes: list1 and list2) and print out the result as illustrated below. The result should be stored in the attribute list3.

```
>mul
Exp1 * Exp2 = +20x^6 +14x^5 +17x^4 +67x^3 +20x^2 +18x^1 +48x^0
```

7. **void PolyCalculator::evaluate(int ExprID, int x)**

A method that takes the expression ID and an integer value and evaluates the polynomial expression for that value. ExprID = 1 evaluates the first polynomial (list1) and ExprID = 2 evaluates the second polynomial (list2).

```
>display
Exp1: +4x^3 +2x^2 +3x^1 +8x^0
Exp2: +5x^3 +1x^2 +6x^0
>evaluate 1,2
p(x) = +4x^3 +2x^2 +3x^1 +8x^0
p(2) = 54
>evaluate 2,3
p(x) = +5x^3 +1x^2 +6x^0
p(3) = 150
```

8. **int PolyCalculator::getDegree(int exprID)**

A method that returns the degree of a polynomial, i.e. the highest exponent in the polynomial.

```
>display
Exp1: +4x^3 +2x^2 +3x^1 +8x^0
Exp2: +5x^3 +1x^2 +6x^0
>getDegree 1
The degree of Exp1 is: 3
>getDegree 2
The degree of Exp2 is: 3
```

9. **void PolyCalculator::read(string filename)**

This method should read and load two polynomial expressions from a file. The existing LinkedLists/expressions must be cleared before the new data is stored into them. The expressions in the file might not necessarily be sorted or simplified. The following is an example of a valid input file.

file1.txt

```
4x^3 +2x^2 +3x^1 +8x^0
5x^3 -3x^2 +4x^2 +6x^0
```

Below is a sample output of the ***read*** command, that invokes the **read** method of PolyCalculator class:

```
>read file1.txt
Exp1: +4x^3 +2x^2 +3x^1 +8x^0
Exp2: +5x^3 +1x^2 +6x^0
```

4 Grading

Description	Score (20)
LinkedList::LinkedList(..)	0.5
LinkedList::insert(..)	3
LinkedList::print(..)	1
LinkedList::removeAll()	1
LinkedList::operator==()	1
LinkedList::~LinkedList(..)	0.5
PolyCalculator::input()	1
PolyCalculator::parse(..)	2
PolyCalculator::display()	1
PolyCalculator::add()	1
PolyCalculator::sub()	1
PolyCalculator::mul()	2
PolyCalculator::evaluate()	1
PolyCalculator::getDegree()	1
PolyCalculator::read(..)	1
Proper code documentation (meaningful comments)	2

You should solve and work **individually** on this assignment. You should submit a single header file named **polycalculator.hpp** to Brightspace and to Gradescope by the deadline. Submissions via any other medium (e.g. via email) are **NOT** acceptable. Passing the Gradescope tests does not guarantee full marks; you are still responsible for thoroughly testing your code. Final grades will be posted on Brightspace.

Note that your program should be implemented in standard C++ and **must be runnable on Linux, Unix, or macOS operating systems**. Make sure to include the required header files in your submission. **It is very important to submit a program that compiles without any errors. Programs with compile-time errors will receive at most 30% of the score.**

Be sure to include meaningful and clear comments that describe the functionality and logic of your code. All comments must be written on **a separate line** (not on the same line as code) so that the auto-grader can properly detect and evaluate them.

Late submissions will be accepted **within 72 hours (3 days) after the deadline**. No submissions will be accepted after that period. All assignments are due at 11:59 pm on the due date. For late submissions, a 5% score will be deducted from the assignment grade per late day.